

Контейнеризация

26 сентября 2025 г.

Наш CI/CD pipeline работает:

- Git push → автоматический деплой через GitHub Actions
- Ansible настраивает nginx на сервере
- deploy.sh копирует файлы приложения

Но есть проблема:

Тестовый и основной сайт находятся на одном сервере и управляются одним nginx.

Почему это проблема?

Наша текущая ситуация:

- **Продакшн** — `app.prafdin.course.prafdin.ru` → `/var/www/demo`
- **Тест** — `app-test.prafdin.course.prafdin.ru` → `/var/www/demo-test`
- Два сервер блока в одном `nginx.conf` на порту 8181
- Один процесс `nginx` обслуживает оба окружения

Проблемы отсутствия изоляции:

- **Перезапуск `nginx`** — ребут теста прерывает продакшн
- **Версия `nginx`** — нельзя тестировать обновления безопасно

Вопрос: как изолировать приложения друг от друга?

Как можно решить проблему изоляции?

- **Отдельные машины** — избыточно для нашего случая
- **Разные процессы nginx** — сложно настраивать
- **Контейнеризация** — индустриальный стандарт

Что такое контейнеризация?

Контейнеризация — технология изоляции приложений с их зависимостями в отдельные исполняемые единицы.

Контейнер — процесс или группа процессов, запущенные в изолированном окружении.

Изоляция на уровне:

- **Процессов** — у каждого контейнера своё дерево процессов
- **Сети** — у каждого контейнера свои сетевые интерфейсы, настройки маршрутизации
- **Ресурсов** — cpu, ram, IO
- **Файловой системы** — у каждого контейнера своя файловая система
- ... — а также много других механизмов ядра

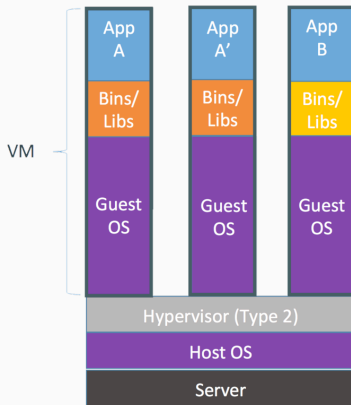
Виртуальные машины

- Полная ОС для каждого приложения
- Большой overhead
- Медленный старт
- Изоляция на уровне железа

Контейнеры

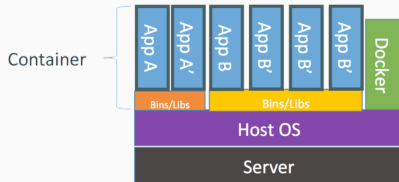
- Общее ядро ОС
- Минимальный overhead
- Быстрый старт (секунды)
- Изоляция на уровне процессов

Контейнеры vs Виртуальные машины



Containers are isolated,
but share OS and, where
appropriate, bins/libraries

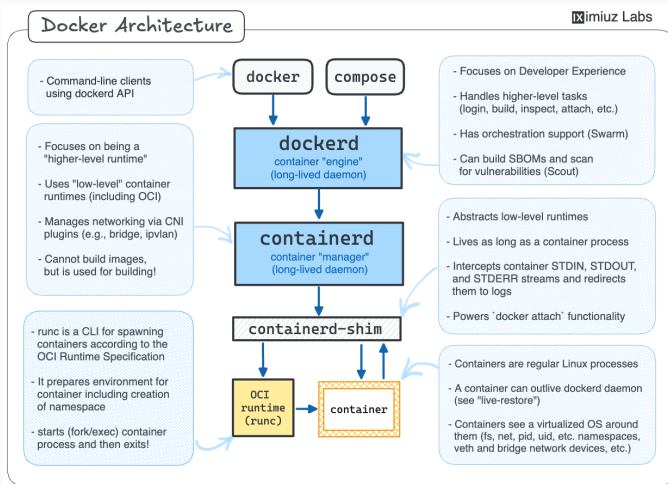
...result is significantly faster deployment,
much less overhead, easier migration,
faster restart



<https://dzone.com/articles/evolution-of-linux-containers-future>

Существует несколько технологий контейнеризации:

- **Docker** — самая популярная платформа контейнеризации
- **Podman** — альтернатива Docker без демона
- **LXC/LXD** — системные контейнеры



Ключевые абстракции Docker:

- **Image** — неизменяемый шаблон для создания контейнеров
- **Container** — выполняющийся экземпляр образа
- **Dockerfile** — инструкции для сборки образа

Принцип работы:

1. Описываем приложение в Dockerfile
2. Собираем образ (image) из Dockerfile
3. Запускаем контейнер из образа

Что изолирует Docker?

Docker создает изоляцию по нескольким уровням:

- **Файловая система** — каждый контейнер видит только свои файлы
- **Процессы** — контейнеры не видят процессы друг друга
- **Сеть** — свои порты, IP-адреса
- **Ресурсы** — изолированное использование CPU, памяти

Два типа registry:

- **Локальный registry** — образы хранятся на локальной машине
- **Удаленный registry** — образы хранятся на удаленном сервере
 - Docker Hub
 - GitLab Registry
 - GitHub Container Registry

Основные операции:

- **Push** — загрузка образа в удаленный registry
- **Pull** — скачивание образа из registry

Что покажем:

1. Dockerfile для сайта
2. Работа с контейнерами через docker cli
3. Использование контейнера в CI/CD пайплайне

Демо-репозиторий:

<https://github.com/prafdin/devops-demo-website/tree/docker-demo>

Было:

- Ansible устанавливаете nginx
- deploy.sh копирует файлы

Стало:

- Ansible устанавливает только docker
- Docker build, push в pipeline
- Запуск на сервере

- **Контейнеризация** — ключевой принцип современной инфраструктуры
- **Контейнеризация решает проблемы** переносимости и конфликтов зависимостей
- **Контейнеризация стандартизирует окружение** и упрощает развертывание

Вопросы?